**Ministry of Education, Culture and Research of the Republic of Moldova**
**Technical University of Moldova**
**Faculty of Computers, Informatics and Microelectronics**
**Department of Software Engineering and Automatic**

**ANDREI ELCISCEV**
**Group FAF-243**

# LABORATORY WORK No. 12
# PRODUCER, CONSUMER AND THE OTHER TOOLS

**Checked by:**
Reiţman Patricia

**Chişinău, 2026**

# Contents

**1 Introduction** **2**

**2 Part 1: Producer and Consumer** **2**

**3 Part 2: Condition Variable** **3**

**4 Part 3: Readers and Writers** **4**

**5 Part 4: Deadlock** **4**

**6 Conclusion** **7**

**A Submitted Source Code** **7**

# 1 Introduction

This laboratory work combines semaphores, mutexes, and condition variables to coordinate POSIX threads. Four experiments cover a bounded buffer, message ordering, reader and writer exclusion, and circular wait. The programs were compiled with `gcc -Wall -pthread` in `/home/user/os-lab12`. The listings contain the actual Linux commands and output. The deliberately blocking variants ran with a five-second timeout and line-buffered output.

```
$ uname -sr
Linux 6.14.0-37-generic
$ gcc -dumpfullversion
13.3.0
```

Listing 1: Linux VM and compiler

# 2 Part 1: Producer and Consumer

```
$ gcc -Wall -pthread -o prodcons prodcons.c
$ ./prodcons
put  1   (in buffer: 1)
put  2   (in buffer: 2)
put  3   (in buffer: 3)
put  4   (in buffer: 4)
            take 1   (in buffer: 3)
put  5   (in buffer: 4)
            take 2   (in buffer: 3)
put  6   (in buffer: 4)
            take 3   (in buffer: 3)
put  7   (in buffer: 4)
            take 4   (in buffer: 3)
put  8   (in buffer: 4)
            take 5   (in buffer: 3)
            take 6   (in buffer: 2)
            take 7   (in buffer: 1)
            take 8   (in buffer: 0)
```

Listing 2: Four-slot buffer with eight items

```
$ sed -n 20,25p prodcons_bad.c
        pthread_mutex_lock(&lock);
        sem_wait(&empty);
        buf[in] = item;
        in = (in + 1) % SIZE;
        count++;
        printf("put  %d   (in buffer: %d)\n", item, count);
$ gcc -Wall -pthread -o prodcons_bad prodcons_bad.c
$ timeout 5 stdbuf -oL ./prodcons_bad; echo "exit code: $?";
put  1   (in buffer: 1)
put  2   (in buffer: 2)
put  3   (in buffer: 3)
put  4   (in buffer: 4)
exit code: 124
```

Listing 3: The altered producer blocks with the buffer full

**Observe 1. What stops the producer at four items?** `sem_wait(&empty)` waits for a free slot. The producer consumes one of four permits per insertion, and the consumer returns a permit with `sem_post(&empty)` after removing an item.

**Observe 2. Who waits at the start?** The consumer must wait at `sem_wait(&full)` if it reaches that call before the first insertion. The initial value of `full` is zero. The producer can proceed immediately because `empty` starts at four.

**Observe 3. What happens if the producer locks before waiting?** The altered program filled four slots and timed out with exit code 124. The producer held the mutex while waiting for an empty slot. The consumer needed that mutex to remove an item and release a slot. The submitted program waits for a slot before acquiring the mutex.

# 3 Part 2: Condition Variable

```
$ gcc -Wall -pthread -o condvar condvar.c
$ ./condvar
B: not ready, I sleep
A: message written
B: I read "hello"
```

Listing 4: Predicate checked in a loop

```
$ sed "s/while (!ready)/if (!ready)/" condvar.c > condvar_if.c
$ gcc -Wall -pthread -o condvar_if condvar_if.c
$ ./condvar_if
B: not ready, I sleep
A: message written
B: I read "hello"
```

Listing 5: Single predicate check in the altered run

**Observe 1. Which variable remembers that the message is ready?** `ready` stores the predicate and the mutex protects it. `changed` is the condition variable used to notify a waiter.

**Observe 2. Did replacing while with if work here?** Yes. The altered run printed the same three lines, ending with the reader printing hello. The writer set `ready = 1` before signaling, and this run had no early wakeup.

**Observe 3. Why check the predicate in a loop?** A condition wait can wake spuriously while `ready` is still zero. The reader reacquires the mutex and checks the predicate again before reading. The submitted program uses the loop.

# 4 Part 3: Readers and Writers

```
$ gcc -Wall -pthread -o readers readers.c
$ ./readers
writer 1 writes 10  (alone)
reader 1 reads 10   (readers at entry: 1)
reader 2 reads 10   (readers at entry: 2)
reader 3 reads 10   (readers at entry: 3)
writer 2 writes 20  (alone)
```

Listing 6: Concurrent readers between exclusive writers

**Observe 1. Which lines let readers share the room?** The first reader executes `if (++readers == 1) sem_wait(&room);`. The last reader executes `if (-readers == 0) sem_post(&room);`. The group holds the room while any reader is inside, so each writer must wait for exclusive access.

**Observe 2. Why does the reader counter need its own lock?** Without `rlock`, updates can race and lose counts. Two readers could both act as the first reader, or a miscount could release the room while readers remain or keep it locked after they leave. The submitted code also takes the diagnostic counter snapshot under `rlock` to avoid an unsynchronized read.

**Observe 3. What if readers keep arriving?** If the active reader count never reaches zero, a waiting writer may never acquire the room. This is writer starvation.

# 5 Part 4: Deadlock

```
$ gcc -Wall -pthread -o deadlock deadlock.c
$ timeout 5 stdbuf -oL ./deadlock; echo "exit code: $?";
two: I have B, I want A
one: I have A, I want B
exit code: 124
```

Listing 7: Circular wait with opposite lock acquisition order

```
$ sed "/^[[:space:]]*sleep(1);/d" deadlock.c > deadlock_fast.c
$ gcc -Wall -pthread -o deadlock_fast deadlock_fast.c
Trial 1
$ timeout 5 stdbuf -oL ./deadlock_fast; echo "exit code: $?"
one: I have A, I want B
one: I have both
two: I have B, I want A
two: I have both
done
exit code: 0
Trial 2
$ timeout 5 stdbuf -oL ./deadlock_fast; echo "exit code: $?"
two: I have B, I want A
one: I have A, I want B
```

```
exit code: 124
Trial 3
$ timeout 5 stdbuf -oL ./deadlock_fast; echo "exit code: $?"
two: I have B, I want A
one: I have A, I want B
exit code: 124
Trial 4
$ timeout 5 stdbuf -oL ./deadlock_fast; echo "exit code: $?"
two: I have B, I want A
two: I have both
one: I have A, I want B
one: I have both
done
exit code: 0
Trial 5
$ timeout 5 stdbuf -oL ./deadlock_fast; echo "exit code: $?"
two: I have B, I want A
two: I have both
one: I have A, I want B
one: I have both
done
exit code: 0
Trial 6
$ timeout 5 stdbuf -oL ./deadlock_fast; echo "exit code: $?"
two: I have B, I want A
two: I have both
one: I have A, I want B
one: I have both
done
exit code: 0
Trial 7
$ timeout 5 stdbuf -oL ./deadlock_fast; echo "exit code: $?"
one: I have A, I want B
one: I have both
two: I have B, I want A
two: I have both
done
exit code: 0
Trial 8
$ timeout 5 stdbuf -oL ./deadlock_fast; echo "exit code: $?"
one: I have A, I want B
one: I have both
two: I have B, I want A
two: I have both
done
exit code: 0
Trial 9
$ timeout 5 stdbuf -oL ./deadlock_fast; echo "exit code: $?"
two: I have B, I want A
one: I have A, I want B
exit code: 124
Trial 10
$ timeout 5 stdbuf -oL ./deadlock_fast; echo "exit code: $?"
two: I have B, I want A
two: I have both
one: I have A, I want B
one: I have both
done
```

```
exit code: 0
```

Listing 8: Ten trials after removing both sleeps

```
$ sed -n 13,35p deadlock.c
    pthread_mutex_lock(&A);
    printf("one: I have A, I want B\n");
    sleep(1);
    pthread_mutex_lock(&B);
    printf("one: I have both\n");
    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);
    return NULL;
}

void *two(void *arg) {
    pthread_mutex_lock(&A);
    printf("two: I have A, I want B\n");
    sleep(1);
    pthread_mutex_lock(&B);
    printf("two: I have both\n");
    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
$ gcc -Wall -pthread -o deadlock deadlock.c
$ timeout 5 stdbuf -oL ./deadlock; echo "exit code: $?";
one: I have A, I want B
one: I have both
two: I have A, I want B
two: I have both
done
exit code: 0
```

Listing 9: Fixed source excerpt and successful execution

**Observe 1. What is the circular wait?** Thread one holds A and waits for B. Thread two holds B and waits for A. The wait-and-ownership cycle is shown below.

```
Thread one -> mutex B -> thread two -> mutex A -> thread one
```

Listing 10: Wait-and-ownership cycle

**Observe 2. Do the ten runs without sleeps always hang?** No. 3 timed out (trials 2, 3, 9) and 7 completed. Removing the sleeps makes the bug depend more on scheduling. A successful run can hide the bug during testing.

**Observe 3. Why does a consistent acquisition order prevent this deadlock?** When both threads acquire A before B, the thread waiting for A holds neither mutex, so there is no circular wait.

# 6 Conclusion

The four-slot buffer preserved FIFO order for all eight items. The condition variable paired a notification with a mutex-protected predicate. The reader group allowed shared access while holding writers out. The deadlock tests showed that timing can hide circular wait, whereas consistent lock ordering removes the cycle between the two threads.

# A Submitted Source Code

```
/* prodcons.c - producer and consumer around a buffer of 4 slots.
 *   empty = free slots (starts at SIZE), full = filled slots (starts at 0), lock
   protects the buffer.
 * Build:  gcc -Wall -pthread -o prodcons prodcons.c
 */
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE  4
#define ITEMS 8

int buf[SIZE];
int in = 0, out = 0, count = 0;     /* count only for printing */
sem_t empty, full;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *producer(void *arg) {
    for (int item = 1; item <= ITEMS; item++) {
        sem_wait(&empty);               /* wait for a free slot */
        pthread_mutex_lock(&lock);
        buf[in] = item;
        in = (in + 1) % SIZE;
        count++;
        printf("put  %d   (in buffer: %d)\n", item, count);
        pthread_mutex_unlock(&lock);
        sem_post(&full);                /* one more filled slot */
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int k = 0; k < ITEMS; k++) {
        sem_wait(&full);                /* wait for something to take */
        pthread_mutex_lock(&lock);
        int item = buf[out];
        out = (out + 1) % SIZE;
        count--;
        printf("            take %d   (in buffer: %d)\n", item, count);
        pthread_mutex_unlock(&lock);
        sem_post(&empty);               /* one more free slot */
        usleep(300000);                 /* the consumer is slow */
    }
    return NULL;
}
```

```
int main(void) {
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_t p, c;
    pthread_create(&c, NULL, consumer, NULL);
    pthread_create(&p, NULL, producer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    sem_destroy(&empty);
    sem_destroy(&full);
    return 0;
}
```

Listing 11: prodcons.c

```
/* condvar.c - the order.c problem, solved with a mutex and a condition variable.
 * Build:  gcc -Wall -pthread -o condvar condvar.c
 */
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>

char message[32];
int ready = 0;                                  /* the condition itself */
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t changed = PTHREAD_COND_INITIALIZER;

void *writer(void *arg) {
    sleep(1);
    pthread_mutex_lock(&lock);
    strcpy(message, "hello");
    ready = 1;
    printf("A: message written\n");
    pthread_cond_signal(&changed);              /* wake one waiter */
    pthread_mutex_unlock(&lock);
    return NULL;
}

void *reader(void *arg) {
    pthread_mutex_lock(&lock);
    while (!ready) {                            /* while, not if: re-check after
    waking */
        printf("B: not ready, I sleep\n");
        pthread_cond_wait(&changed, &lock);     /* unlocks, sleeps, locks again */
    }
    printf("B: I read \"%s\"\n", message);
    pthread_mutex_unlock(&lock);
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&b, NULL, reader, NULL);
    pthread_create(&a, NULL, writer, NULL);
    pthread_join(a, NULL);
```

```
    pthread_join(b, NULL);
    return 0;
}
```

Listing 12: condvar.c

```
/* readers.c - readers and writers with two semaphores.
 *   Many readers may read together; a writer must be alone.
 * Build:  gcc -Wall -pthread -o readers readers.c
 */
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define READERS 3
#define WRITERS 2

int value = 0;                  /* the shared data */
int readers = 0;                /* how many are reading now */
sem_t rlock;                    /* protects the counter readers */
sem_t room;                     /* 1 = the room is free for a writer */

void *reader(void *arg) {
    long id = (long)arg;
    sem_wait(&rlock);
    if (++readers == 1)         /* the first reader in locks out the writers */
        sem_wait(&room);
    int inside = readers;      /* snapshot under the counter lock */
    sem_post(&rlock);

    printf("reader %ld reads %d   (readers at entry: %d)\n", id, value, inside);
    usleep(200000);

    sem_wait(&rlock);
    if (--readers == 0)         /* the last reader out lets the writers in */
        sem_post(&room);
    sem_post(&rlock);
    return NULL;
}

void *writer(void *arg) {
    long id = (long)arg;
    sem_wait(&room);            /* alone in the room */
    value += 10;
    printf("writer %ld writes %d  (alone)\n", id, value);
    usleep(200000);
    sem_post(&room);
    return NULL;
}

int main(void) {
    sem_init(&rlock, 0, 1);
    sem_init(&room, 0, 1);
    pthread_t r[READERS], w[WRITERS];
    pthread_create(&w[0], NULL, writer, (void *)1L);
    for (long i = 0; i < READERS; i++)
```

```
        pthread_create(&r[i], NULL, reader, (void *)(i + 1));
    pthread_create(&w[1], NULL, writer, (void *)2L);
    for (int i = 0; i < READERS; i++) pthread_join(r[i], NULL);
    for (int i = 0; i < WRITERS; i++) pthread_join(w[i], NULL);
    return 0;
}
```

Listing 13: readers.c

```
/* deadlock.c - two threads, two mutexes, both taken in A-then-B order.
 * Build:  gcc -Wall -pthread -o deadlock deadlock.c
 * Run:    timeout 5 ./deadlock; echo "exit code: $?"      (expect exit code 0)
 */
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

pthread_mutex_t A = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t B = PTHREAD_MUTEX_INITIALIZER;

void *one(void *arg) {
    pthread_mutex_lock(&A);
    printf("one: I have A, I want B\n");
    sleep(1);
    pthread_mutex_lock(&B);
    printf("one: I have both\n");
    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);
    return NULL;
}

void *two(void *arg) {
    pthread_mutex_lock(&A);
    printf("two: I have A, I want B\n");
    sleep(1);
    pthread_mutex_lock(&B);
    printf("two: I have both\n");
    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, one, NULL);
    pthread_create(&t2, NULL, two, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("done\n");
    return 0;
}
```

Listing 14: deadlock.c